

# GIẢI THÍCH CHI TIẾT HỆ THỐNG TASK GENERATION

AI-Powered Requirement Analysis & Task Generator

AI Project - Requirement Analyzer

Ngày 2 tháng 4 năm 2026

# Mục lục

|          |                                                         |           |
|----------|---------------------------------------------------------|-----------|
| <b>1</b> | <b>GII THIU H THNG</b>                                  | <b>2</b>  |
| 1.1      | Task Generation System là gì? . . . . .                 | 2         |
| 1.2      | Quy Trình X Lý - Pipeline . . . . .                     | 2         |
| <b>2</b> | <b>CÁC STAGE CHI TIT</b>                                | <b>4</b>  |
| 2.1      | Stage 1: Segmentation (Tách Tài Liu) . . . . .          | 4         |
| 2.1.1    | Mc ích . . . . .                                        | 4         |
| 2.1.2    | u Vào & u Ra . . . . .                                  | 4         |
| 2.1.3    | Ví D . . . . .                                          | 5         |
| 2.2      | Stage 2: Requirement Detection (Phát Hin) . . . . .     | 5         |
| 2.2.1    | Mc ích . . . . .                                        | 5         |
| 2.2.2    | Cách Hot ng . . . . .                                   | 5         |
| 2.3      | Stage 3: Enrichment (Phân Loi) . . . . .                | 6         |
| 2.3.1    | Mc ích . . . . .                                        | 6         |
| 2.3.2    | Multi-Class Classifiers . . . . .                       | 6         |
| 2.3.3    | Ví D Enrichment . . . . .                               | 7         |
| 2.4      | Stage 4: Task Generation (To Task) . . . . .            | 7         |
| 2.4.1    | 3 Modes To Task . . . . .                               | 7         |
| 2.4.2    | So Sánh 3 Modes . . . . .                               | 10        |
| 2.5      | Stage 5: Post-Processing . . . . .                      | 11        |
| 2.5.1    | Các Bc . . . . .                                        | 11        |
| <b>3</b> | <b>TASK SCHEMA - C TRÚC D LIU</b>                       | <b>12</b> |
| 3.1      | GeneratedTask Model . . . . .                           | 12        |
| <b>4</b> | <b>PIPELINE CODE - HOW IT WORKS</b>                     | <b>14</b> |
| 4.1      | Main Pipeline Class . . . . .                           | 15        |
| 4.2      | Usage Example . . . . .                                 | 16        |
| <b>5</b> | <b>USER STORIES VÀ TASKS TRONG H THNG</b>               | <b>17</b> |
| 5.1      | Task Generation Feature - User Stories . . . . .        | 17        |
| 5.1.1    | US1: To Requirement Document . . . . .                  | 17        |
| 5.1.2    | US2: Generate Tasks from Requirements . . . . .         | 18        |
| 5.1.3    | US3: Review & Edit Generated Tasks . . . . .            | 19        |
| 5.1.4    | US4: Export Tasks to Jira . . . . .                     | 19        |
| 5.2      | Related Backend Tasks . . . . .                         | 20        |
| 5.2.1    | Task: Implement Requirement Detector ML Model . . . . . | 20        |
| 5.2.2    | Task: Implement Enrichment Classifiers . . . . .        | 21        |
| 5.2.3    | Task: Implement LLM Integration . . . . .               | 22        |

---

|          |                                                 |           |
|----------|-------------------------------------------------|-----------|
| 5.2.4    | Task: Implement Model-Based Generator . . . . . | 23        |
| <b>6</b> | <b>KIN TRÚC &amp; TECHNICAL DETAILS</b>         | <b>24</b> |
| 6.1      | System Architecture . . . . .                   | 25        |
| 6.2      | Component Dependencies . . . . .                | 26        |
| 6.3      | Data Flow Example . . . . .                     | 27        |
| <b>7</b> | <b>CÓ GÌ VUI VN V H THNG</b>                    | <b>28</b> |
| 7.1      | Advantages . . . . .                            | 28        |
| 7.2      | Limitations & Future Work . . . . .             | 28        |
| <b>8</b> | <b>SUMMARY</b>                                  | <b>29</b> |

# Chương 1

## GIỚI THIỆU HỆ THỐNG

### 1.1 Task Generation System là gì?

**Task Generation System** là một hệ thống AI powerpoint được xây dựng để tự động \*\*chuyển đổi tài liệu yêu cầu (Requirements Document) thành các Task/User Story có cấu trúc\*\*.

#### Mục đích chính:

- Tự động phân tích tài liệu yêu cầu
- Tách yêu cầu từ tài liệu thô
- Tạo Task/User Story theo chuẩn Agile
- Phân loại, ước tính Story Points
- Sẵn sàng upload lên Jira

### 1.2 Quy Trình Xử Lý - Pipeline

Hệ thống hoạt động theo một pipeline 5 bước:

[RAW DOCUMENT]



[STAGE 1: SEGMENTATION]

Tách tài liệu thành các câu/đoạn



[STAGE 2: REQUIREMENT DETECTION]

Phát hiện câu nào là yêu cầu thực tế



[STAGE 3: ENRICHMENT]

Phân loại: Type, Priority, Domain, Role



[STAGE 4: TASK GENERATION]

Tạo Task/User Story chi tiết



[STAGE 5: POST-PROCESSING]

Loại bỏ trùng lặp, định dạng lại



[STRUCTURED TASKS OUTPUT]

(JSON - Ready for Jira)

## Chương 2

# CÁC STAGE CHI TIẾT

### 2.1 Stage 1: Segmentation (Tách Tài Liệu)

#### 2.1.1 Mục Đích

Chia tài liệu yêu cầu thành các đơn vị xử lý nhỏ (câu, đoạn).

#### 2.1.2 Đầu Vào & Đầu Ra

| Input                                   | Output                                                                                                                                                              |
|-----------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Raw document<br>(TXT, MD,<br>DOCX, PDF) | List of Sentences with metadata: <ul style="list-style-type: none"><li>• Text content</li><li>• Line number</li><li>• Doc offset</li><li>• Section header</li></ul> |

### 2.1.3 Ví Dụ

#### Input Document:

Requirements for Hotel Management System:

1. The system must allow customers to search for available rooms.
2. Check-in and check-out dates are required.

#### Output (Segmented):

Sentences:

- [0] "The system must allow customers to search for available rooms."  
- line: 3  
- section: "Requirements for Hotel Management System"
- [1] "Check-in and check-out dates are required."  
- line: 5  
- section: "Requirements for Hotel Management System"

## 2.2 Stage 2: Requirement Detection (Phát Hiện)

### 2.2.1 Mục Đích

Phát hiện **which sentences are actual requirements** và **filter out noise** (tiêu đề, giải thích, v.v.).

### 2.2.2 Cách Hoạt Động

Sử dụng **trained ML classifier** để dự đoán xác suất một câu là yêu cầu:

**Requirement Detector Model:**

- **Input:** Câu text
- **Feature Extraction:** TF-IDF vectorization
- **Model:** Logistic Regression / SVM
- **Output:** Probability [0-1]
- **Threshold:**  $> 0.5$  = Yêu cầu (tunable)

**Ví dụ dự đoán:**

"The system must allow customers to search"→  
Probability: 0.92 (IS Requirement)

"Hotel Management System"→  
Probability: 0.15 (NOT Requirement)

"Customers can modify their bookings"→  
Probability: 0.87 (IS Requirement)

## 2.3 Stage 3: Enrichment (Phân Loại)

### 2.3.1 Mục Đích

Gắn \*\*metadata vào mỗi requirement\*\*: Type, Priority, Domain, Role.

### 2.3.2 Multi-Class Classifiers

| Label    | Classes                                            | Ví Dụ                         |
|----------|----------------------------------------------------|-------------------------------|
| Type     | Functional, Security, Interface, Data, Performance | "User can login"→ Functional  |
| Priority | Critical, High, Medium, Low                        | "Must encrypt data"→ Critical |
| Domain   | ecommerce, iot, healthcare, finance, general       | "Payment processing"→ Finance |
| Role     | Backend, Frontend, QA, DevOps, BA, Security        | "Database setup"→ DevOps      |



### 2.3.3 Ví Dụ Enrichment

**Input (Requirement):**

```
"The system must encrypt customer  
payment information in transit and at rest"
```

**Output (Enriched):**

```
GeneratedTask {  
  title: "Encrypt customer payment data",  
  type: "security", ←Classifier 1  
  priority: "Critical", ←Classifier 2  
  domain: "finance", ←Classifier 3  
  role: "Backend", ←Classifier 4  
  ...  
}
```

## 2.4 Stage 4: Task Generation (Tạo Task)

### 2.4.1 3 Modes Tạo Task

Hệ thống hỗ trợ **3** cách khác nhau để tạo task:

**Mode 1: Template-Based (Nhanh & Consistent)****Đặc điểm:**

- Nhanh nhất (milliseconds)
- Sử dụng templates cố định
- Output consistent
- Không cần API

**Cách hoạt động:**

1. Extract entities từ requirement  
(subject, action, object, condition)

2. Match với template tương ứng

3. Fill-in template với entities

Template ví dụ (Functional):

Title: "Implement {action} for {object}"

Description: "The system needs to {action}  
{object} {condition}."

AC: - User can {action} {object}  
- System validates input  
- Error handling

**Mode 2: LLM-Based (Linh Hoạt & Natural)****Đặc điểm:**

- Sử dụng OpenAI/Anthropic API
- Natural language output
- Context-aware (hiểu ngữ cảnh)
- Tồn chi phí API
- Chậm hơn template

**Cách hoạt động:**

LLMTaskGenerator↓

1. Gửi requirement đến LLM (GPT-4, Claude) ở vi prompt structured
2. LLM tạo task output theo schema JSON
3. Validate response
4. Return GeneratedTask

Prompt ví dụ:

"Given this requirement: {requirement}"

Generate a Jira-like task with:

- Title (action-oriented)
- Description
- 3-5 Acceptance Criteria

Output as JSON matching this schema: {...}"

### Mode 3: Model-Based (Production - No APIs)

#### Đặc điểm:

- Production ready
- Không cần API keys
- Nhanh & offline
- Trained on project data
- Data privacy (on-device)

#### Cách hoạt động:

ModelBasedTaskGenerator↓

1. Load trained ML models  
(requirement detector, enrichers)
2. Extract entities using spaCy NLP
3. Apply generation patterns  
(NOT hardcoded templates)
4. Generate natural variations  
using trained models
5. Return GeneratedTask

Models trained on:

- Project requirements dataset
- Historical tasks
- Requirement patterns

### 2.4.2 So Sánh 3 Modes

| Aspect         | Template  | LLM        | Model    |
|----------------|-----------|------------|----------|
| Speed          |           |            |          |
| Flexibility    |           |            |          |
| Cost           | Free      | \$\$\$     | Free     |
| API needed     | No        | Yes        | No       |
| Training data  | Hardcoded | In-context | External |
| Natural output |           |            |          |

## 2.5 Stage 5: Post-Processing

### 2.5.1 Các Bước

1. **Deduplication:** Loại bỏ task trùng lặp
2. **Validation:** Kiểm tra required fields
3. **Normalization:** Chuẩn hóa format
4. **Filtering:** Loại bỏ low-confidence tasks
5. **Sorting:** Sắp xếp theo priority

## Chương 3

# TASK SCHEMA - CỬ TRÚC DỮ LIỆU

### 3.1 GeneratedTask Model

Mỗi task sinh ra có cấu trúc Pydantic schema như sau:

```
class GeneratedTask(BaseModel):  
    # ID & Identifiers  
    task_id: str = UUID  
    epic: Optional[str]  
    module: Optional[str]  
  
    # Core Content  
    title: str # "Implement user login"  
    description: str # Detailed paragraph  
    acceptance_criteria: List[str] # 3-5 AC items  
  
    # Classification Labels  
    type: str # functional, security, interface...  
    priority: str # Critical, High, Medium, Low  
    domain: str # ecommerce, healthcare, finance...  
    role: str # Backend, Frontend, QA, DevOps...  
    labels: List[str] # Additional tags  
  
    # Estimation  
    story_points: Optional[int] # 1, 2, 3, 5, 8, 13, 21  
    estimated_hours: Optional[float]  
    confidence: float # [0.0 - 1.0]  
    complexity: Optional[str] # Simple, Medium, Complex  
  
    # Source Tracking  
    source: Optional[TaskSource]  
    - sentence: str # Original requirement  
    - section: Optional[str]  
    - doc_offset: [start, end]  
    - line_number: int  
  
    # Dependencies  
    dependencies: List[str] # task IDs this depends on  
  
    # Metadata  
    generated_at: datetime  
    generator_version: str
```





## Chương 4

# PIPELINE CODE - HOW IT WORKS

### 4.1 Main Pipeline Class

```
class TaskGenerationPipeline:
    """
    Main orchestrator combining all stages
    """

    def __init__(
        self,
        generator_mode: str = "template",
        # "template" (fast)
        # "llm" (flexible, needs API)
        # "model" (production, trained)
        llm_provider: Optional[str] = None,
        llm_model: Optional[str] = None,
        llm_api_key: Optional[str] = None
    ):
        self.mode = generator_mode

        # Initialize components
        self.segmenter = get_segmenter()
        self.detector = get_detector() # Req detection
        self.enricher = get_enrichment_pipeline()

        # Choose generator based on mode
        if mode == "template":
            self.generator = TemplateTaskGenerator()
        elif mode == "llm":
            self.generator = LLMTaskGenerator(
                provider=llm_provider,
                model=llm_model,
                api_key=llm_api_key
            )
        elif mode == "model":
            self.generator = ModelBasedTaskGenerator()

    def generate_tasks(
        self,
        document: str,
        max_tasks: int = 100,
```

## 4.2 Usage Example

```
from requirement_analyzer.task_gen import get_pipeline

# Initialize with different modes
pipeline_fast = get_pipeline(mode="template")
pipeline_flex = get_pipeline(
    mode="llm",
    llm_provider="openai",
    llm_model="gpt-4o-mini"
)
pipeline_prod = get_pipeline(mode="model")

# Use pipeline
document = """
Hotel Management System Requirements:

1. The system must allow customers to search
   for available rooms by date range.
2. Customers can view room details including
   price, amenities, and availability.
3. System must send confirmation email
   after booking is confirmed.
4. Payment must be encrypted using SSL/TLS.
"""

# Generate tasks
result = pipeline_fast.generate_tasks(
    document,
    max_tasks=20,
    min_confidence=0.6
)

# Access results
print(f"Generated {result.total_count} tasks")
for task in result.tasks:
    print(f"\n{task.title}")
    print(f"  Priority: {task.priority}")
    print(f"  Story Points: {task.story_points}")
    print(f"  Type: {task.type}")
    print(f"  Role: {task.role}")
    print(f"  Confidence: {task.confidence:.2%}")
    print(f"  AC: {len(task.acceptance_criteria)}")

# Export to Jira or JSON
result.export_to_jira(project_key="HMS")
result.export_to_json("generated_tasks.json")
```

## Chương 5

# USER STORIES VÀ TASKS TRONG HỆ THỐNG

### 5.1 Task Generation Feature - User Stories

#### 5.1.1 US1: Tạo Requirement Document

Title: Upload requirement document

As a Project Manager

I want to upload requirement documents

So that I can generate tasks automatically

Acceptance Criteria:

User can upload TXT, MD, DOCX, PDF files

System validates file format

Shows upload progress

Displays file content preview

Error handling for invalid files

Type: Functional

Priority: High

Story Points: 3

### 5.1.2 US2: Generate Tasks from Requirements

Title: Generate tasks with multiple modes

As a Project Manager

I want to generate tasks using multiple generation modes (Template/LLM/Model)  
So that I can choose best approach for my project needs

Acceptance Criteria:

Scenario 1: Template Mode

Select "Fast Template Generation"  
System uses predefined templates  
Tasks generated within 2 seconds  
Output is consistent across runs

Scenario 2: LLM Mode

Select "Flexible LLM Generation"  
Configure LLM provider (OpenAI/Anthropic)  
System generates natural task descriptions  
Request/response logged for audit

Scenario 3: Model Mode

Select "Production Model"  
System uses locally trained models  
No API keys required  
Fast processing without internet

Type: Functional

Priority: High

Story Points: 8

Complexity: Complex

### 5.1.3 US3: Review & Edit Generated Tasks

Title: Review and edit generated tasks

As a Project Manager  
I want to review, edit, or delete  
generated tasks  
So that I can ensure quality before  
importing to Jira

Acceptance Criteria:

- Display tasks in table/list view
- Can edit title, description, AC
- Can change priority/type/story points
- Can delete unwanted tasks
- Show confidence score for each task
- Bulk operations (select multiple)
- Changes saved to local storage

Type: Interface

Priority: High

Story Points: 5

### 5.1.4 US4: Export Tasks to Jira

Title: Export generated tasks to Jira

As a Project Manager  
I want to export tasks to Jira directly  
So that I can quickly populate project board

Acceptance Criteria:

- Authenticate with Jira API
- Select target Jira project
- Map task fields to Jira fields
- Support custom fields
- Show import progress
- Handle duplicate prevention
- Create tasks in batches
- Error logging for failed imports

Type: Integration

Priority: High

Story Points: 8

Dependencies: [US3, Jira API setup]

## 5.2 Related Backend Tasks

### 5.2.1 Task: Implement Requirement Detector ML Model

Title: Train requirement detection classifier

Description:

Develop and train ML model to classify whether a sentence is an actual requirement

Technical Details:

- Model type: Logistic Regression / SVM
- Features: TF-IDF vectors
- Training data: 1000+ labeled sentences
- Classes: [Requirement, Non-requirement]
- Target accuracy: >90%

Acceptance Criteria:

Model trained on dataset  
Cross-validation accuracy >90%  
Model exported and serialized  
Inference time <10ms per sentence  
Model integrated with pipeline  
Unit tests passing

Type: Data/ML

Priority: Critical

Story Points: 13

Complexity: Complex

## 5.2.2 Task: Implement Enrichment Classifiers

Title: Train multi-class enrichment classifiers

Description:

Build 4 separate classifiers for:

1. Type classifier (7 classes)  
functional, security, interface, data,  
performance, integration, documentation
2. Priority classifier (4 classes)  
Critical, High, Medium, Low
3. Domain classifier (6 classes)  
ecommerce, iot, healthcare, finance,  
education, general
4. Role classifier (6 classes)  
Backend, Frontend, QA, DevOps, BA, Security

Acceptance Criteria:

Each classifier accuracy >85%  
Fast inference (<100ms per sentence)  
Models serialized and versioned  
Integrated in enrichment pipeline  
Fallback to defaults if confidence low  
Pipeline tests passing

Type: Data/ML

Priority: High

Story Points: 21

Complexity: Very Complex

### 5.2.3 Task: Implement LLM Integration

Title: Integrate with LLM API (OpenAI/Anthropic)

Description:

Implement `LLMTaskGenerator` class to use external LLM APIs for task generation

Technical Requirements:

- Support OpenAI (GPT-4, GPT-4o-mini)
- Support Anthropic (Claude-3-Haiku)
- Structured JSON output
- Streaming support
- Error handling & retries
- Cost tracking (API usage)
- Rate limiting

Acceptance Criteria:

OpenAI integration working  
Anthropic integration working  
Structured prompt engineering  
JSON validation  
Caching implemented  
Error messages clear  
Cost calculation accurate  
Integration tests passing

Type: Integration

Priority: High

Story Points: 8



## 5.2.4 Task: Implement Model-Based Generator

Title: Implement production model-based generator

Description:

Build ModelBasedTaskGenerator using trained ML models for on-device task generation

Technical Details:

- Load trained models from disk
- Extract entities with spaCy NLP
- Generate task variations
- No external API required

Acceptance Criteria:

Models loaded correctly  
Entity extraction working  
Task generation deterministic  
Performance >1000 tasks/min  
Memory usage <500MB  
All AC met  
E2E tests passing  
Documentation complete

Type: Backend

Priority: High

Story Points: 13



## Chương 6

# KIẾN TRÚC & TECHNICAL DETAILS

### 6.1 System Architecture

FastAPI Web Server  
(routes/api endpoints)

Task Gen Pipeline

File Upload  
Processing

Segmenter  
(sentence splitting)

Detector  
(ML classifier)  
models/requirement\_detector\_model.joblib

Enricher  
(4 classifiers)  
- type\_classifier  
- priority\_classifier  
- domain\_classifier  
- role\_classifier

Generator (3 modes)  
TemplateGenerator (fast)  
LLMGenerator (flexible)  
ModelGenerator (production)

PostProcessor  
(dedup, validation)

Output: JSON/Jira

## 6.2 Component Dependencies

| Component    | Dependencies         | Purpose               |
|--------------|----------------------|-----------------------|
| Segmenter    | nlTK, spaCy          | Sentence tokenization |
| Detector     | scikit-learn, joblib | ML classification     |
| Enrichers    | scikit-learn, joblib | Multi-class labeling  |
| Template Gen | spaCy, regex         | Pattern matching      |
| LLM Gen      | openai/anthropic     | External API calls    |
| Model Gen    | spaCy, joblib        | On-device inference   |

## 6.3 Data Flow Example

EXAMPLE: User uploads hotel\_requirements.md

### 1. UPLOAD STAGE

File: hotel\_requirements.md  
Size: 50 KB  
Status: Uploaded

### 2. SEGMENTATION

Input: Raw text (1200 words)  
Process: Split into sentences  
Output: 98 sentences with metadata

### 3. REQUIREMENT DETECTION

Processed: 98 sentences  
Detected: 74 are requirements (>0.5 confidence)  
Filtered out: 24 non-requirement sentences

### 4. ENRICHMENT (74 requirements)

Type Distribution:  
- Functional: 45 (60%)  
- Security: 12 (16%)  
- Data: 10 (13%)  
- Performance: 5 (7%)  
- Interface: 2 (3%)

Priority Distribution:  
- Critical: 8  
- High: 34  
- Medium: 28  
- Low: 4

### 5. TASK GENERATION (Mode: Template)

Time: 280ms  
Tasks generated: 74  
Avg confidence: 0.85

### 6. POST-PROCESSING

Removed duplicates: 3  
Final tasks: 71  
Ready for Jira export

### 7. OUTPUT

Format: JSON  
Size: 125 KB  
Status: Ready for import

# Chương 7

## CÓ GÌ VUI VẪN VỀ HỆ THỐNG

### 7.1 Advantages

- **Tự động hóa:** Không cần tạo task thủ công
- **Consistent:** Tất cả task cùng format, structure
- **Flexible:** 3 generation modes với trade-offs khác nhau
- **Smart:** ML classifiers learn từ dữ liệu
- **Jira-ready:** Export directly to Jira/JSON
- **Scalable:** Handle 1000+ requirements
- **Traceable:** Track source sentence cho mỗi task

### 7.2 Limitations & Future Work

- Không perfect 100% - cần human review
- Language-specific - models trained for English
- Requires clean input - garbage in = garbage out
- ML models need retraining với new patterns
- Future: Multi-language support
- Future: Custom model training per-project
- Future: Story point auto-estimation (ML)
- Future: Dependency detection between tasks
- Future: Real-time feedback & refinement

# Chương 8

## SUMMARY

**Task Generation System** của bạn là một **production-grade AI system** sau:

1. **Input:** Raw requirement documents (TXT, MD, DOCX, PDF)
2. **Process:** 5-stage pipeline (Segment → Detect → Enrich → Generate → Post-process)
3. **3 Generation Modes:**
  - Template (fast, consistent, no API)
  - LLM (flexible, natural, needs API)
  - Model (production, trained, no API)
4. **Output:** Structured tasks (JSON/Jira)
5. **Benefits:** Automation, consistency, scalability, quality

**Cách sử dụng:**

```
# Fast generation
tasks = pipeline.generate_tasks(doc, mode="template")

# Flexible generation
tasks = pipeline.generate_tasks(doc, mode="llm",
                                provider="openai")

# Production
tasks = pipeline.generate_tasks(doc, mode="model")
```